

ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE (SRS)

CÓDIGO DE DOCUMENTO: SGMM-SRS-001

VERSIÓN: 1.0

FECHA: 14 de Junio de 2026

PROYECTO: Sistema de Gestión Integrado para Mercados Municipales (SGMM)

DIRECTOR DE PROYECTO: Univ. Daniel Moya

1. Introducción

1.1. Propósito

Este documento define los requisitos funcionales y no funcionales para la construcción del Producto Mínimo Viable (MVP) del Sistema de Gestión Integrado para Mercados Municipales (SGMM). Sirve como contrato técnico entre el equipo de desarrollo y la Dirección de Ingresos de la Alcaldía de Sucre.

1.2. Alcance del Sistema

El SGMM digitalizará la recaudación de tasas de asentamiento en los mercados Central, Campesino y Entre Ríos. Comprende dos plataformas conectadas mediante una API REST:

- 1. Plataforma Web (Administración):** Orientada al control del padrón gremial, emisión de credenciales (Códigos QR) y fiscalización de ingresos.
- 2. Aplicación Móvil (Operación en Campo):** Orientada a los inspectores municipales para la lectura de QR, verificación de deudas y registro de pagos in situ, con capacidad de operar sin conexión a internet.

2. Descripción General

2.1. Perfiles de Usuario (Actores)

Actor	Nivel de Acceso	Descripción del Rol
Administrador Global	<i>Total</i>	<i>Funcionario de jerarquía. Visualiza reportes financieros de todos los mercados, gestiona usuarios (inspectores) y audita las bases de datos.</i>
Operador de Mercado	<i>Medio</i>	<i>Administrador de un mercado específico. Registra a los comerciantes, asigna los espacios físicos y genera los códigos QR impresos.</i>
Inspector de Campo	<i>Restringido</i>	<i>Usuario de la App Móvil. Su única función es escanear los QR en los puestos, registrar el pago físico y sincronizar los datos al finalizar el turno.</i>

2.2. Restricciones del Sistema

- *El sistema no procesará pagos electrónicos (tarjetas, transferencias) en esta versión; el inspector seguirá recibiendo dinero en efectivo, pero el registro del cobro será estrictamente digital.*
- *El servidor central operará sobre infraestructura basada en distribuciones Linux gestionada por el municipio.*

3. Requerimientos Funcionales (RF)

Los requerimientos funcionales definen las acciones específicas que el sistema debe ejecutar, priorizados para el MVP.

Módulo Administrativo (Web)

ID	Nombre	Descripción
-----------	---------------	--------------------

RF-01	<i>Gestión de Padrón (CRUD)</i>	<i>El sistema debe permitir registrar, editar, visualizar y dar de baja a comerciantes, capturando datos personales, sector de venta y rubro.</i>
RF-02	<i>Gestión de Puestos</i>	<i>El sistema debe permitir asignar un espacio físico (dimensiones y ubicación) a un comerciante registrado.</i>
RF-03	<i>Generador de QR</i>	<i>Al registrar un puesto, el sistema generará automáticamente un Código QR único asociado al ID del puesto y del comerciante, exportable en formato PDF para impresión.</i>
RF-04	<i>Panel de Recaudación</i>	<i>El sistema mostrará un Dashboard con gráficos estadísticos de los cobros realizados, filtrables por fecha, mercado e inspector responsable.</i>
RF-05	<i>Gestión de Usuarios</i>	<i>El Administrador debe poder crear credenciales de acceso para los Inspectores, asignándoles un mercado específico.</i>

Módulo de Fiscalización (Aplicación Móvil)

ID	Nombre	Descripción
RF-06	<i>Autenticación Móvil</i>	<i>El inspector debe ingresar a la app mediante usuario y contraseña asignados.</i>
RF-07	<i>Lectura de Código QR</i>	<i>La app debe acceder a la cámara del dispositivo para escanear el Código QR pegado en el puesto del comerciante.</i>
RF-08	<i>Visualización de Estado</i>	<i>Tras escanear el QR, la app mostrará el nombre del titular, el rubro y la deuda actual acumulada (tasa diaria).</i>

ID	Nombre	Descripción
RF-09	<i>Registro de Pago</i>	<i>El inspector debe poder confirmar el pago presionando un botón, lo que reducirá la deuda a cero para la fecha correspondiente.</i>
RF-10	Motor de Sincronización	<i>La app debe guardar los cobros en la base de datos local del teléfono. Al presionar "Sincronizar", enviará los registros empaquetados a la base de datos central.</i>

4. Requerimientos No Funcionales (RNF)

Estos requisitos definen los atributos de calidad, rendimiento y arquitectura del sistema.

4.1. Disponibilidad y Arquitectura Offline-First

- **RNF-01 (Operación sin Red):** *La aplicación móvil de los inspectores debe permitir escanear códigos QR y registrar pagos de manera fluida y sin bloqueos de interfaz aunque el dispositivo no cuente con señal Wi-Fi ni datos móviles (crítico para las zonas techadas del Mercado Campesino).*
- **RNF-02 (Sincronización Bidireccional):** *El proceso de sincronización debe descargar el padrón actualizado al celular al iniciar el turno, y subir los cobros al servidor al finalizar la jornada.*

4.2. Rendimiento (Performance)

- **RNF-03 (Velocidad Móvil):** *El escaneo del QR y la visualización del estado de cuenta en modo offline no debe demorar más de **2 segundos**.*
- **RNF-04 (Límite de Sincronización):** *El envío de un lote de 500 registros de cobro desde la app al servidor debe completarse en menos de **10 segundos** bajo una red Wi-Fi estándar.*

4.3. Accesibilidad y Diseño UI/UX

- **RNF-05 (Alto Contraste):** *La interfaz de la aplicación móvil debe utilizar una paleta de colores de alto contraste que permita su lectura directa bajo la luz solar intensa.*

- **RNF-06 (Prevención de Errores):** Los botones de "Confirmar Pago" deben ser amplios (mínimo 48x48 dp) e incluir alertas de confirmación háptica (vibración) para evitar que los inspectores marquen pagos por accidente en entornos concurridos.

4.4. Seguridad

- **RNF-07 (Autenticación):** Toda comunicación entre la aplicación móvil, la plataforma web y la base de datos central debe estar protegida mediante tokens JWT (JSON Web Tokens).
- **RNF-08 (Auditoría):** Todo pago registrado en la base de datos debe almacenar obligatoriamente el ID del inspector que realizó el cobro, la fecha y la hora exacta del dispositivo.

5. Arquitectura y Stack Tecnológico Propuesto

Para garantizar el cumplimiento de los requerimientos y optimizar el tiempo de entrega de 6 meses, se implementará la siguiente pila tecnológica:

- **Backend y API REST:** Framework Spring Boot (Java) o .NET (C#) para la lógica de negocio y procesamiento de alta concurrencia.
- **Frontend Web (Administración):** React.js / Next.js para un panel interactivo y dinámico.
- **Aplicación Móvil (Inspectores):** React Native, garantizando compilación nativa para dispositivos Android con uso de SQLite local para satisfacer el requisito Offline-First.
- **Bases de Datos:** PostgreSQL para el motor de base de datos relacional central en los servidores de la Alcaldía.
- **Infraestructura y Despliegue:** Contenedores Docker para empaquetar el backend y la web, asegurando portabilidad y fácil despliegue en el sistema operativo Linux base de los servidores municipales.